

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL [berapa] – [nama modul]



NAMA: Fauzi Romadhoni

NIM: 103132400025

KELAS: Sisd_05_A

PROGRAM STUDI SAINS DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Pemrograman Berorientasi Objek (Object Oriented Programming/OOP) merupakan metode pemrograman yang menggabungkan data dan fungsi ke dalam suatu objek.

Dalam OOP terdapat beberapa konsep utama, yaitu **abstraksi, enkapsulasi, inheritance, polymorphism, dan komunikasi antar objek.**

B. Guided

1. [Field]

```
[public class Circle {  
    public static final double PI= 3.14159;  
    public static double radiansToDegrees(double rads) {  
        return rads * 180 / PI;  
    }  
    public double r;  
    public double area() {  
        return PI * r * r;  
    }  
    public double circumference() {  
        return 2 * PI * r;  
    }  
}]
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ PraktikumLingkaran ---  
Jari-jari: 7.0  
Luas (Area): 153.93791  
Keliling (Circumference): 43.98226  
Konversi 1 radian ke derajat: 57.295827908797776  
-----  
[ BUILD SUCCESS ]
```

2. [Method]

```
[  
//file TestPass.java  
class TestPass {  
    int i,j;  
    TestPass(int a,int b) {  
        i = a;  
        j = b;  
    }  
    //passed by value dengan parameter berupa tipe data primitif  
    void calculate(int m,int n) {  
        m = m*10;  
        n = n/2;  
    }  
    //passed by reference dengan berupa tipe data class
```

```
void calculate(TestPass e) {  
    e.i = e.i*10;  
    e.j = e.j/2;  
}  
}]
```

Output :

```
] --- exec:3.6.3:exec (default-cli) @ method ---  
    Nilai sebelum passed by value :  
    x = 10  
    y = 20  
    Nilai sesudah passed by value :  
    x = 10  
    y = 20  
    Nilai sebelum passed by reference :  
    z.i = 50  
    z.j = 100  
    Nilai sesudah passed by reference :  
    z.i = 500  
    z.j = 50  
-----  
[ BUILD SUCCESS ]
```

3. [Keyword "this"]

```
[class Lagu {  
    private String pencipta;  
    private String judul;  
  
    public void IsiParam(String judul, String pencipta) {  
        this.judul = judul;  
        this.pencipta = pencipta;  
    }  
  
    public void cetakKeLayar() {  
        if (judul == null && pencipta == null)  
            return;  
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);  
    }  
}  
  
public class keyword {  
    public static void main(String[] args) {  
        Lagu a = new Lagu();  
        a.IsiParam("God Will Make A Way", "Don Moen");  
        a.cetakKeLayar();  
    }  
}]
```

Output :

```
] --- exec:3.6.3:exec (default-cli) @ keyword ---  
- Judul : God Will Make A Way, pencipta : Don Moen  
-----  
[ BUILD SUCCESS ]
```

C. Unguided

1. Soal Unguided 1

- [1. Membuat class bernama PengolahSuhu dengan *instance field* untuk array suhu dan *class field* (NILAI_KOSONG) bernilai -1.0 yang bersifat tetap dan dibagikan ke semua object.
2. Membuat method untuk menampilkan data, mencari indeks data yang kosong, mengisi data kosong tersebut menggunakan teknik imputasi *time-series* (rata-rata dari hari sebelum dan sesudahnya), serta menghitung rata-rata suhu.
3. Menjalankan alur program di dalam method main dan menganalisis mengapa array suhuHarian di main ikut berubah nilainya secara otomatis karena sifat *reference* pada array di Java.

Jawaban :

```
[import java.util.Arrays;  
  
class PengolahSuhu {  
    private double[] suhuHarian;  
    public static final double NILAI_KOSONG = -1.0;  
  
    public PengolahSuhu(double[] suhuHarian) {  
        this.suhuHarian = suhuHarian;  
    }  
  
    public void tampilkanData() {  
        for (int i = 0; i < suhuHarian.length; i++) {  
            if (suhuHarian[i] == NILAI_KOSONG) {  
                System.out.println("Hari " + (i + 1) + " : (kosong)");  
            } else {  
                System.out.println("Hari " + (i + 1) + " : " + suhuHarian[i] + "°C");  
            }  
        }  
    }  
  
    public int cariIndexKosong() {  
        for (int i = 0; i < suhuHarian.length; i++) {  
            if (suhuHarian[i] == NILAI_KOSONG) {
```

```

        return i;
    }
}
return -1;
}

public void isiDataKosong() {
    int indexKosong = cariIndexKosong();
    if (indexKosong != -1) {
        suhuHarian[indexKosong] = (suhuHarian[indexKosong - 1] +
suhuHarian[indexKosong + 1]) / 2.0;
    }
}

public double hitungRataRata() {
    double total = 0;
    for (double suhu : suhuHarian) {
        total += suhu;
    }
    return total / suhuHarian.length;
}
}

public class Main {
    public static void main(String[] args) {
        double[] suhuHarian = { 30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9 };

        PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);

        System.out.println("=== Data Suhu Awal ===");
        pengolah.tampilkanData();

        int indexKosong = pengolah.cariIndexKosong();
        System.out.println("\nIndex hari kosong (dimulai dari 0): " + indexKosong);

        pengolah.isiDataKosong();

        System.out.println("\n=== Data Suhu Setelah Pengisian ===");
        pengolah.tampilkanData();

        double rataRata = pengolah.hitungRataRata();
        System.out.printf("\nRata-rata : %.2f°C\n", rataRata);

        System.out.println("\nIsi array suhuHarian di main setelah isiDataKosong()
dijalankan:");
        System.out.println(Arrays.toString(suhuHarian));

        /*
        * Penjelasan Mengapa Array di main Ikut Berubah:
        * Array di Java bertipe reference (referensi). Ketika constructor
        'PengolahSuhu'

```

```

        * dipanggil dengan melewati variabel 'suhuHarian', object tidak membuat
        salinan (copy)
        * dari array tersebut, melainkan menyimpan referensi ke alamat memori
        array yang sama.
        * Akibatnya, setiap perubahan nilai yang dilakukan oleh method di dalam
        object
        * (seperti 'isiDataKosong') akan langsung memengaruhi array asli yang ada
        di dalam method 'main'.
        */
    }
}
}]

```

Output :

```

=== Data Suhu Awal ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : (kosong)
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : 29.1°C
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Rata-rata : 29,39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]

```

```
[ BUILD SUCCESS ]
```

Penjelasan :

[Struktur dan Aturan Class: Kita diminta membuat class bernama PengolahSuhu yang memuat dua jenis field, yaitu instance field (suhuHarian untuk array suhu) dan class field (NILAI_KOSONG = -1.0 sebagai nilai penanda data rusak/kosong yang bersifat tetap dan dibagikan ke semua object).

D. Kesimpulan

[Praktikum Modul 3 Pemrograman Berorientasi Objek ini berhasil membuktikan bahwa penerapan konsep OOP sederhana—tanpa memerlukan inheritance, overriding, ataupun abstract class—dapat diorganisasikan dengan baik melalui pembuatan class yang modular dan memiliki tugas spesifik.]